

chapter 7



主讲人：周瑜

7.1 图的基本概念

7.2 存储结构

7.3 图的遍历

7.4 图的连通性

7.5 图的应用

7.6 最短路径

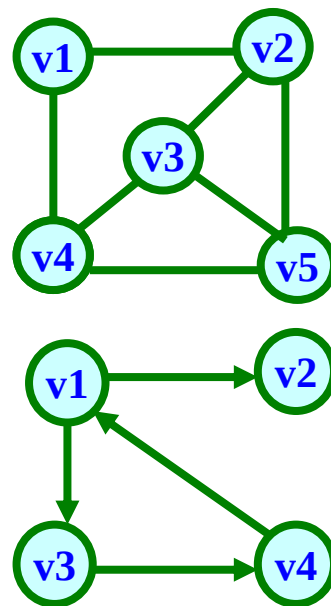
图：记为 $G=(V, E)$

V =vertex
 E =edge

其中： V 是 G 的顶点集合，是有穷非空集；
 E 是 G 的边集合，是有穷集。

问：当 $E(G)$ 为空时，图 G 存在否？

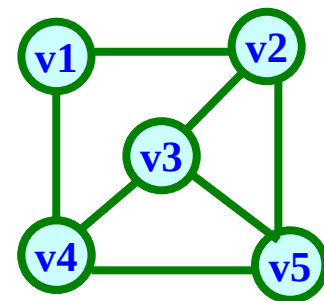
答：还存在！但此时图 G 只有顶点而没有边。



有向图：图 G 中的每条边都是有方向的；

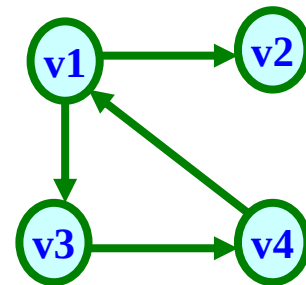
无向图：图 G 中的每条边都是无方向的；

完全图：图 G 任意两个顶点都有一条边相连接；



若 n 个顶点的无向图有 $n(n-1)/2$ 条边，称为无向完全图

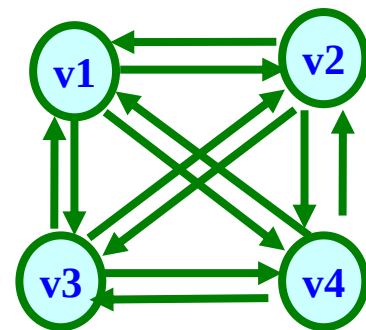
若 n 个顶点的有向图有 $n(n-1)$ 条边，称为有向完全图



证明：

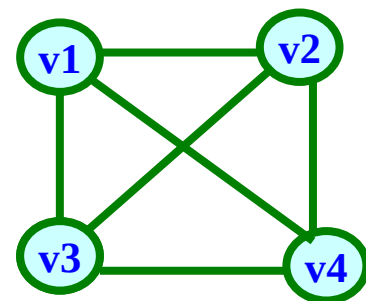
① 完全有向图有 $n(n-1)$ 条边。

证明：若是完全有向图，则 n 个顶点中的每个顶点都有一条弧指向其它 $n-1$ 个顶点，因此总边数 $=n(n-1)$

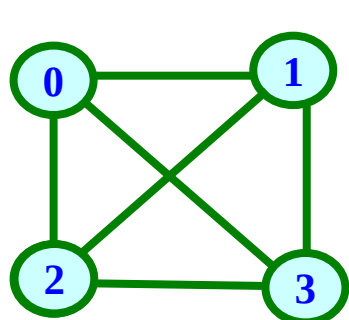


② 完全无向图有 $n(n-1)/2$ 条边。

证明：从①可以直接推论出无向完全图的边数——因为无方向，两弧合并为一边，所以边数减半，总边数为 $n(n-1)/2$ 。



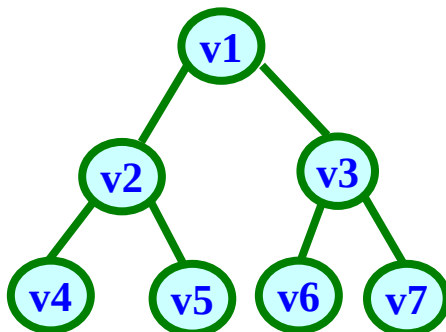
例：判断下列4种图形各属什么类型？



G1

无向**完全图**

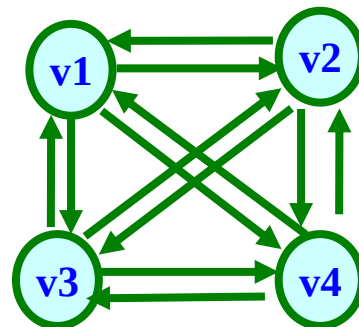
$n(n-1)/2$ 条边



无向图(树)



有向图



有向**完全图**

$n(n-1)$ 条边

G1的顶点集合为 $V(G1)=\{0,1,2,3\}$

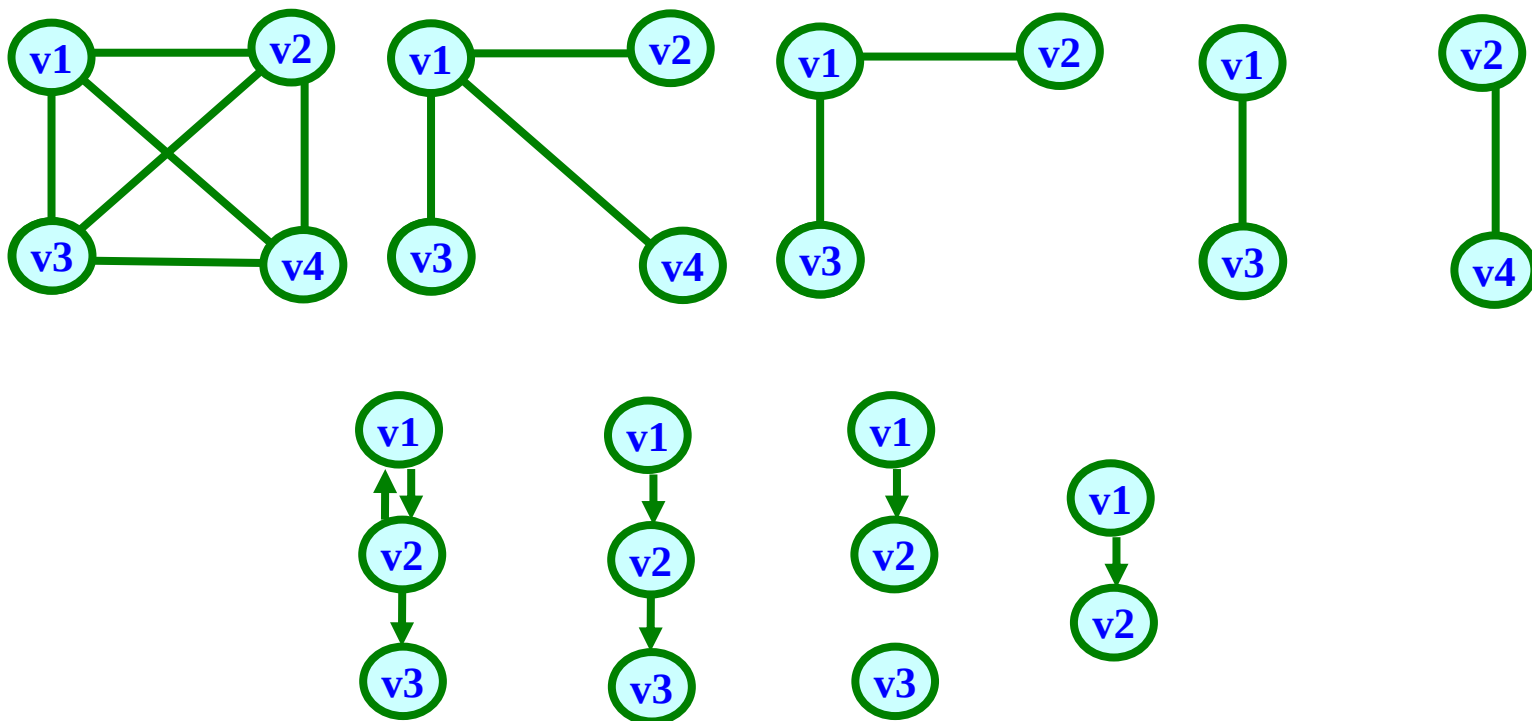
边集合为 $E(G1)=\{(0,1),(0,2),(0,3),(1,2),(1,3),(2,3)\}$

稀疏图： 边较少的图。通常边数远少于 $n\log n$

稠密图： 边很多的图。 $\left\{ \begin{array}{l} \text{无向图中，边数接近 } n(n-1)/2 \\ \text{有向图中，边数接近 } n(n-1) \end{array} \right.$

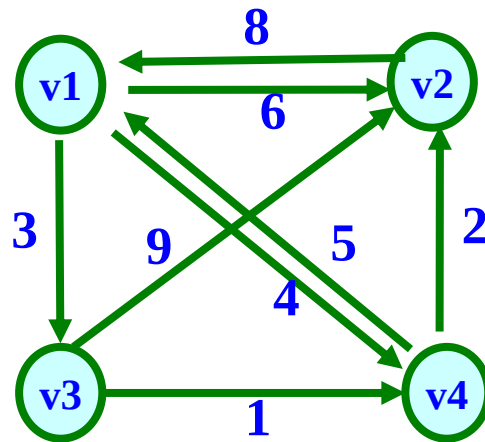
子图：设有两个图 $G = (V, E)$ 和 $G' = (V', E')$ 。

若 $V' \subseteq V$ 且 $E' \subseteq E$ ，则称图 G' 是图 G 的子图。

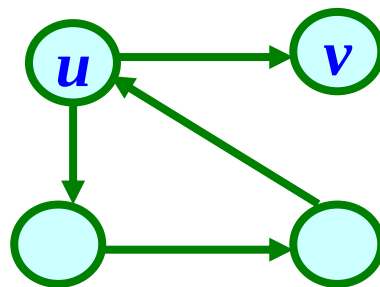


带权图：即边上带权的图。其中权是指每条边可以标上具有某种含义的数值（即与边相关的数）。

网 络：→带权图



邻接点：若 (u, v) 是 $E(G)$ 中的一条边，则称 u 与 v 互为邻接顶点。



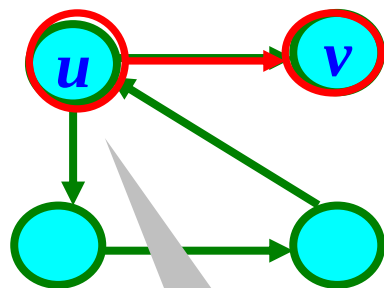
弧头和弧尾：有向边 (u, v) 称为弧，边的始点 u 叫弧尾，终点 v 叫弧头。

度：顶点 v 的度是与它相关联的边的条数。记作 $TD(v)$ 。

在有向图中，顶点的度等于该顶点的入度与出度之和。

入度： 顶点 v 的入度是以 v 为终点的有向边的条数，记作 $ID(v)$ ；

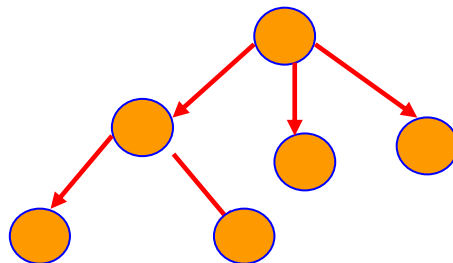
出度： 顶点 v 的出度是以 v 为始点的有向边的条数，记作 $OD(v)$ 。



u 的入度=?
 u 的出度=?

问：当有向图中仅1个顶点的入度为0, 其余顶点的入度均为1, 此时是何形状?

答：是树！而且是一棵有向树！

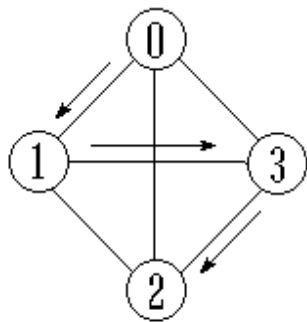


路径： 在图 $G=(V, E)$ 中, 若从顶点 v_i 出发, 沿一些边经过一些顶点 $v_{p1}, v_{p2}, \dots, v_{pm}$, 到达顶点 v_j 。则称顶点序列 $(v_i, v_{p1}, v_{p2}, \dots, v_{pm}, v_j)$ 为从顶点 v_i 到顶点 v_j 的路径。它经过的边 $(v_i, v_{p1}), (v_{p1}, v_{p2}), \dots, (v_{pm}, v_j)$ 应当是属于 E 的边。

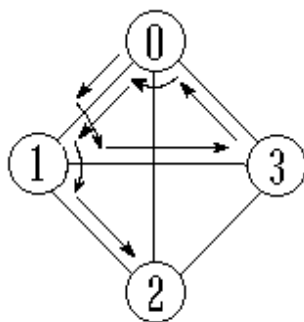
路径长度： 非带权图的路径长度是指此**路径上边的条数**；
带权图的路径长度是指**路径上各边的权之和**。

简单路径：路径上各顶点 $v_1, v_2, \dots, v_m$ 均不互相重复。

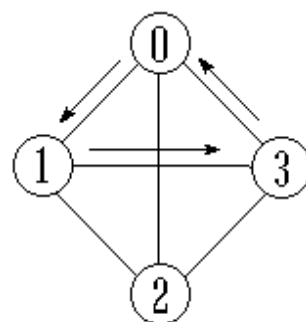
回路：若路径上第一个顶点 v_1 与最后一个顶点 v_m 重合，则称这样的路径为回路或环。



(a) 简单路径

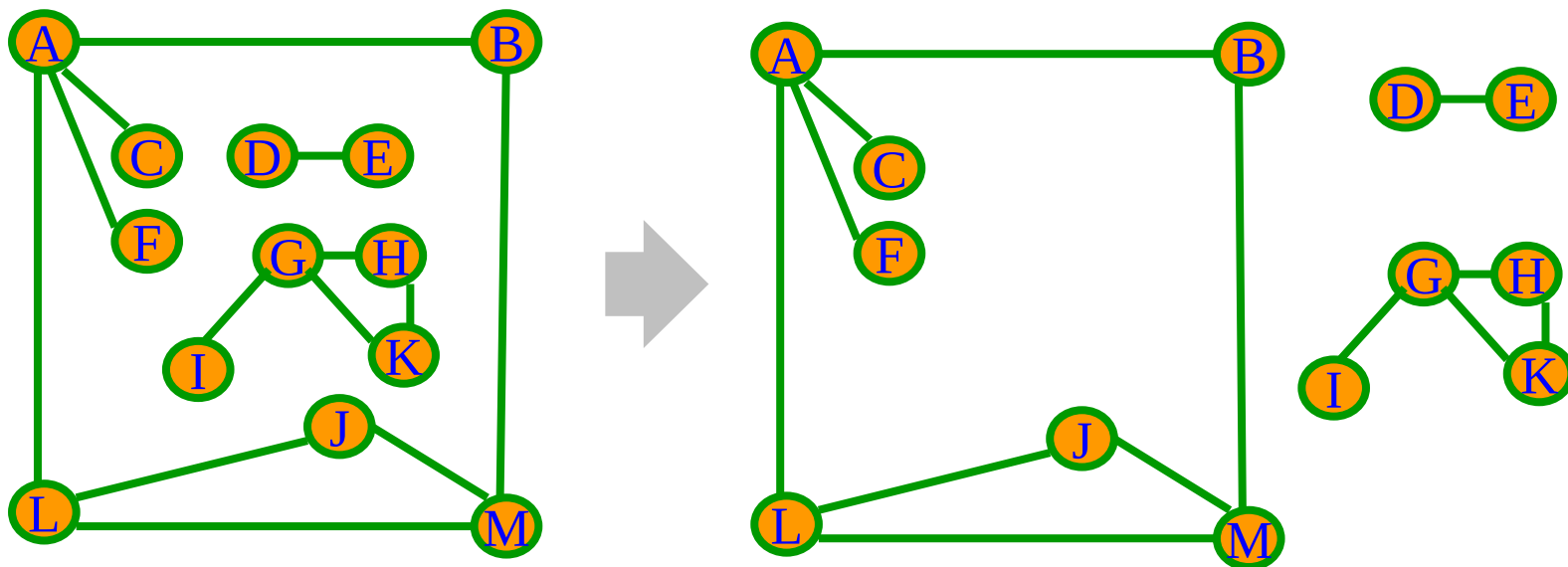


(b) 非简单路径

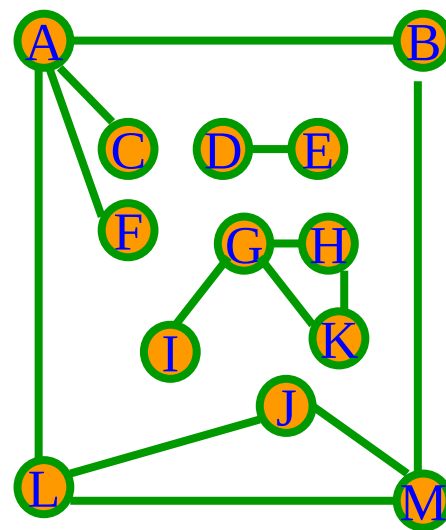


(c) 回路

连通图：在**无向图**中，若从顶点 v_1 到顶点 v_2 有路径，则称顶点 v_1 与 v_2 是连通的。如果图中任意一对顶点都是连通的，则称此图是连通图。**非连通图**的极大连通子图叫做**连通分量**。

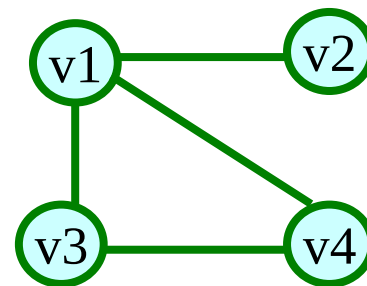


强连通图： 在**有向图**中，若对于每一对顶点 v_i 和 v_j ，都存在一条从 v_i 到 v_j 和从 v_j 到 v_i 的路径，则称此图是强连通图。



非强连通图的极大强连通子图叫做强连通分量。

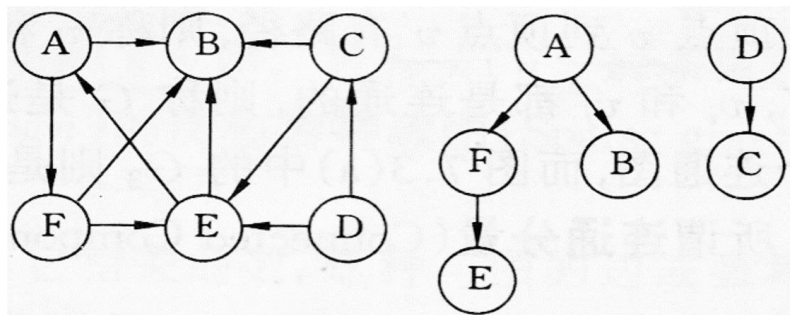
生成树：极小连通子图，它含有图中全部 n 个顶点，但只有 $n-1$ 条边。



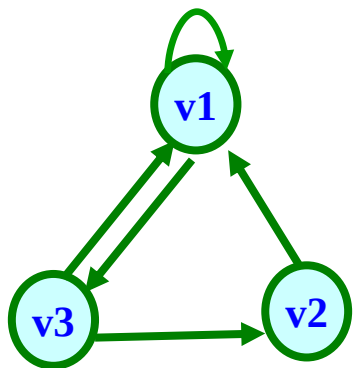
如果在生成树上添加1条边，必定构成一个环。

若图中有 n 个顶点，却少于 $n-1$ 条边，必为非连通图。

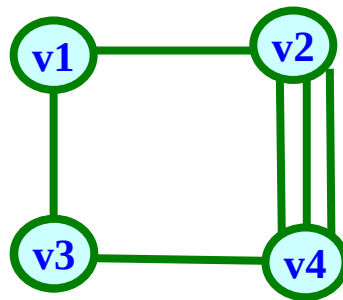
生成森林：若干棵生成树的集合，含全部顶点，但构成这些树的边或弧是最少的。



有两类图形不在本章讨论之列：



带自身环的图



多重图

图的抽象数据类型

图是一种数据元素间存在**多对多关系**的数据结构加上一组基本操作构成的抽象数据类型。

ADT Graph{

数据对象：V是具有相同特性的数据元素的集合，称为顶点集。

数据关系： $R = \{ E \}$

$E = \{ \langle v, w \rangle \mid v, w \in V \text{ 且 } P(v, w),$

顶点对 $\langle v, w \rangle$ 称为边或弧， $P(v, w)$ 定义了 $\langle v, w \rangle$ 的意义或信息}

基本操作：

初始化操作

`CreateGraph(&G, num);`

操作结果：构造一个顶点数为 num ，边数为0的图。

销毁操作

`DestroyGraph(&G);`

初始条件：图 G 存在。操作结果：销毁图 G 。

引用型操作：

若 $\langle v, w \rangle \in G$ ，则称 w 为 v 的邻接点，
若 $(v, w) \in G$ ，则称 w 和 v 互为邻接点。

$\text{FirstAdjVex}(G, v)$;

初始条件：图 G 存在， v 是 G 中某个顶点。

操作结果：返回 v 的第一个邻接点。若该顶点在 G 中没有邻接点，则返回“空”。

若 v 有多个邻接点，则在图的存储结构建立之后，其邻接点之间的相对次序也就自然形成了。

$\text{NextAdjVex}(G, v, w)$;

初始条件：图 G 存在， v 是 G 中某个顶点， w 是 v 的邻接顶点。

操作结果：返回 v 的（相对于 w 的）下一个邻接点。若 w 是 v 的最后一个邻接点，则返回“空”。

加工型操作：

`InsertEdge(&G, v, w);`

初始条件：图 G 存在， v 和 w 是 G 中两个顶点。

操作结果：在 G 中增添以 v, w 为顶点的边

`DeleteEdge(&G, v, w);`

初始条件：图 G 存在， v 和 w 是 G 中两个顶点。

操作结果：在 G 中删除以 v, w 为顶点的边。

} ADT Graph

7.1 图的基本概念

7.2 图的存储结构

7.3 图的遍历

7.4 图的连通性

7.5 图的应用

7.6 最短路径

图的特点：非线性结构 ($m : n$)

顺序存储结构：多个顶点，无序，仅用顶点坐标无法表达相互关系，但可用数组描述元素间关系。

邻接矩阵

链式存储结构：可用多重链表

- 邻接表
- 十字链表
- 邻接多重表

1. 邻接矩阵 (数组) 表示法
2. 邻接表 (链式) 表示法
3. 十字链表 表示法
4. 邻接多重表 表示法

各种表示法成立的原则：存入
电脑后能惟一复原

1. 邻接矩阵（数组）表示法

表示各个顶点之间关系

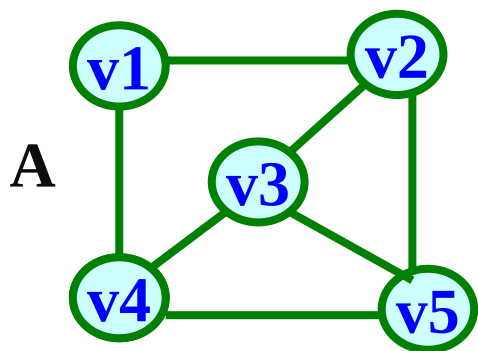
① 建立一个顶点表和一个邻接矩阵。

记录各个顶点信息

② 设图 $A=(V, E)$ 有 n 个顶点，则图的邻接矩阵是一个二维数组 $A.Edge[n][n]$ ，定义为：

$$A.Edge[i][j] = \begin{cases} 1, & \text{如果 } \langle i, j \rangle \in E \text{ 或者 } (i, j) \in E \\ 0, & \text{否则} \end{cases}$$

例1：下面无向图的邻接矩阵如何表示？



顶点表：

(v1 v2 v3 v4 v5)

邻接矩阵：

$A.Edge =$

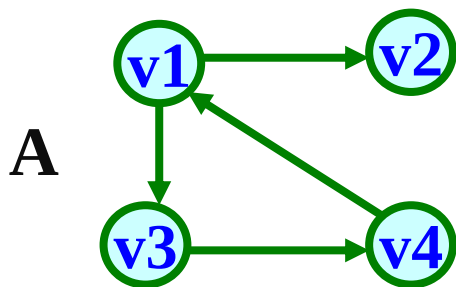
	v1	v2	v3	v4	v5
v1	0	1	0	1	0
v2	1	0	1	0	1
v3	0	1	0	1	1
v4	1	0	1	0	1
v5	0	1	1	1	0

分析1：无向图的邻接矩阵是对称的；

分析2：顶点*i*的度=第*i*行(列)中1的个数；

特别：完全图的邻接矩阵中，对角元素为0，其余全1。

例2：下面有向图的邻接矩阵如何表示？



顶点表:

邻接矩阵:

$A.Edge =$

	(v1 v2 v3 v4)
v1	0 1 1 0
v2	0 0 0 0
v3	0 0 0 1
v4	1 0 0 0

注：在有向图的邻接矩阵中，

第*i*行含义：以结点 v_i 为尾的弧（即出度边）；

第*i*列含义：以结点 v_i 为头的弧（即入度边）。

分析1：有向图的邻接矩阵可能是不对称的。

分析2：顶点 v_i 的出度 = 第 i 行元素之和

$$OD(v_i) = \sum_j A.Edge[i][j]$$

顶点 v_i 的入度=第 i 列元素之和。

$$ID(v_i) = \sum_j A.Edge[j][i]$$

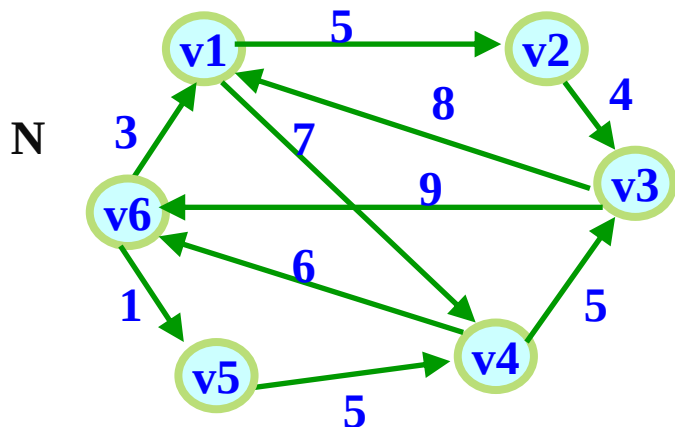
顶点的度 = 第 i 行元素之和 + 第 i 列元素之和

$$\text{即： } TD(v_i) = OD(v_i) + ID(v_i)$$

例3：有权图（即网络）的邻接矩阵如何表示？

定义： $A.Edge[i][j] = \begin{cases} W_{ij} & \langle v_i, v_j \rangle \text{ 或 } (v_i, v_j) \in VR \\ \infty & \text{无边（弧）} \end{cases}$

以有向网为例：



顶点表：

邻接矩阵：

N.Edge =

	(v1 v2 v3 v4 v5 v6)
v1	∞ 5 ∞ 7 ∞ ∞
v2	∞ ∞ 4 ∞ ∞ ∞
v3	8 ∞ ∞ ∞ ∞ 9
v4	∞ ∞ 5 ∞ ∞ 6
v5	∞ ∞ ∞ 5 ∞ ∞
v6	3 ∞ ∞ ∞ 1 ∞

邻接矩阵法

优点： 容易实现图的操作，如：求某顶点的度、判断顶点之间是否有边（弧）、找顶点的邻接点等等。

n 个顶点需要 $n \times n$ 个单元存储边（弧）；空间效率为 $O(n^2)$ 。

缺点： 对稀疏图而言尤其浪费空间。

图的邻接矩阵的存储结构

注：用两个数组分别存储顶点表和邻接矩阵

```
#define INFINITY INT_MAX      //最大值 $\infty$ 
```

```
#define MAX_VERTEX_NUM 20 //假设的最大顶点数
```

```
Typedef enum {DG, DN, UG, UN } GraphKind; //有向/无向图, 有向/无向网
```

```
Typedef struct ArcCell{      //弧（边）结点的定义
    VRType adj;      //顶点间关系，无权图取1或0；有权图取权值类型
    InfoType *info;    //该弧相关信息的指针
} ArcCell, AdjMatrix [ MAX_VERTEX_NUM ] [MAX_VERTEX_NUM ];
```

```
Typedef struct{              //图的定义
    VertexType vexs [MAX_VERTEX_NUM ] ; //顶点表，用一维向量即可( $n$ )
    AdjMatrix arcs;          //邻接矩阵 $n*n$ 
    int Venum, arcnum; //顶点总数 $n$ , 弧（边）总数 $e$ 
    GraphKind kind; //图的种类标志
} MGraph;
```

对于 n 个顶点的图或网，空间效率= $O(n^2)$

例：用邻接矩阵生成无向网的算法

```

Status CreateUDN(MGraph &G){    //无向网的构造，用邻接矩阵表示
scanf(&G.vexnum, &G.arcnum, &IncInfo); //输入总顶点数n、总弧数e和信息
for(i=0;i<G.vexnum; ++i) scanf(&G.vexs[i] ); //输入n个顶点值，存入一维向量

for(i=0; i<G.vexnum; ++i)  adj=∞,info=NULL //对邻接矩阵n*n个单元初始化
for(j=0;j<G.vexnum; ++j)  G.arcs[i][j]={INFINITY, NULL};

for(k=0;k<G.arcnum; ++k){    //给邻接矩阵有关单元赋初值(循环次数=弧数e
    scanf(&v1, &v2, &w);    //输入弧的两顶点以及对应权值
    i=LocateVex(G,v1); j=LocateVex(G,v2);    //找到两顶点在矩阵中的位置(n次)
    G.arcs[i][j].adj=w;    //输入对应权值
    if(IncInfo) Input(*G.arcs[i][j].info); //如果弧有信息则填入
    G.arcs[j][i] = G.arcs[i][j]; //无向网是对称矩阵
}

return OK;
} // CreateUDN

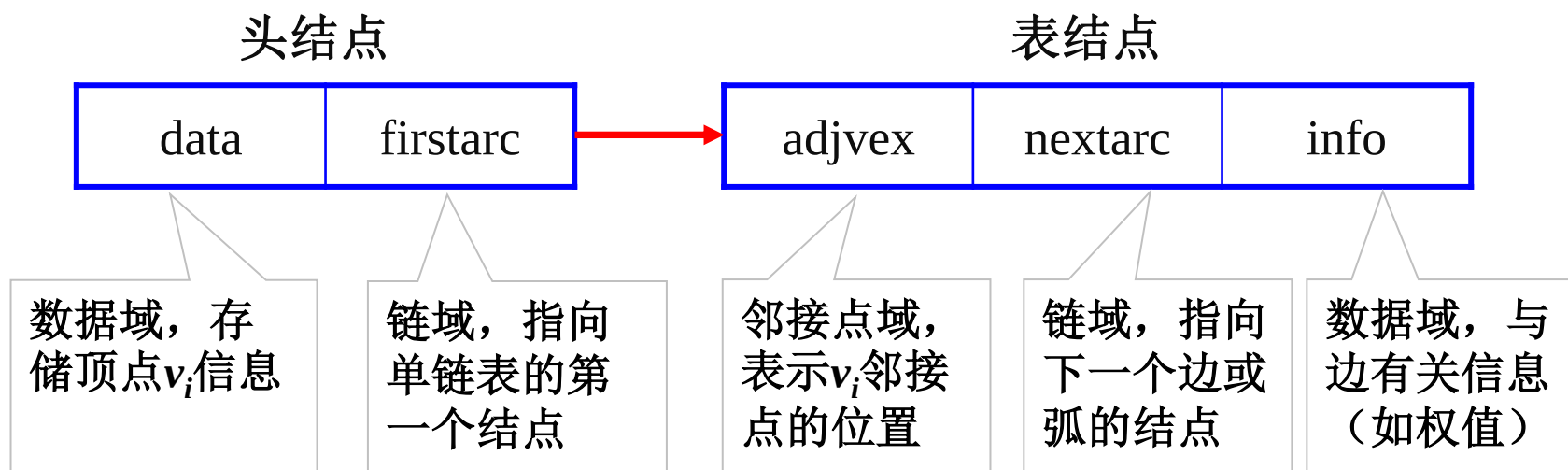
```

对于 n 个顶点 e 条弧的网，建网时间效率 = $O(n+n^2+e*n)$

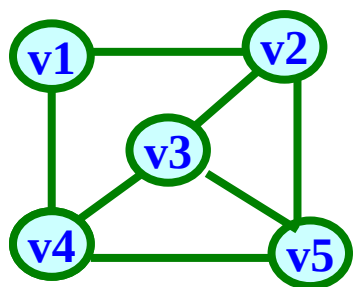
2. 邻接表（链式）表示法

- ① 对每个顶点 v_i 建立一个单链表，把与 v_i 有关联的边的信息（即度或出度边）链接起来，表中每个结点都设为3个域
- ② 每个单链表还应当附设一个头结点（设为2个域），存 v_i 信息；
- ③ 每个单链表的头结点另外用顺序存储结构存储。

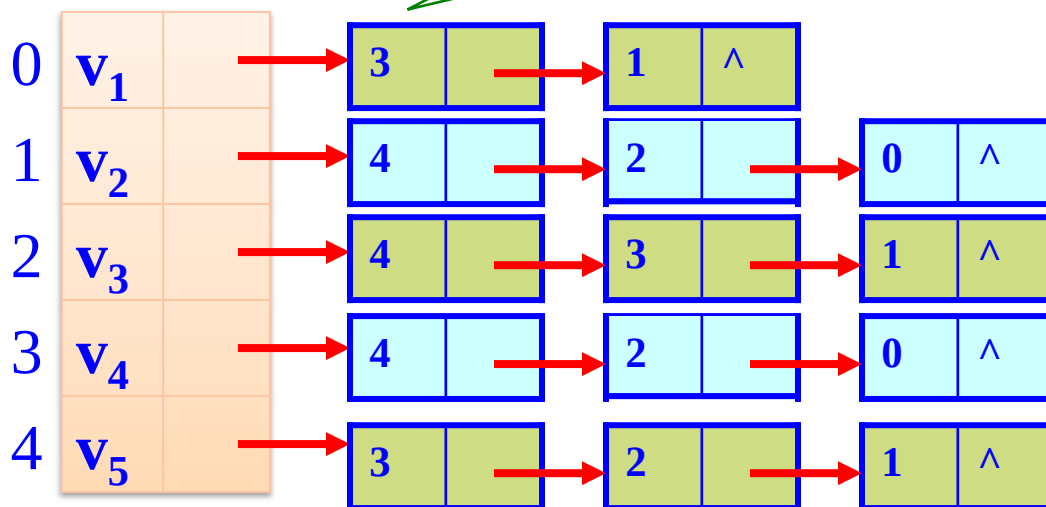
2. 邻接表（链式）表示法



例1：无向图的邻接表如何表示？



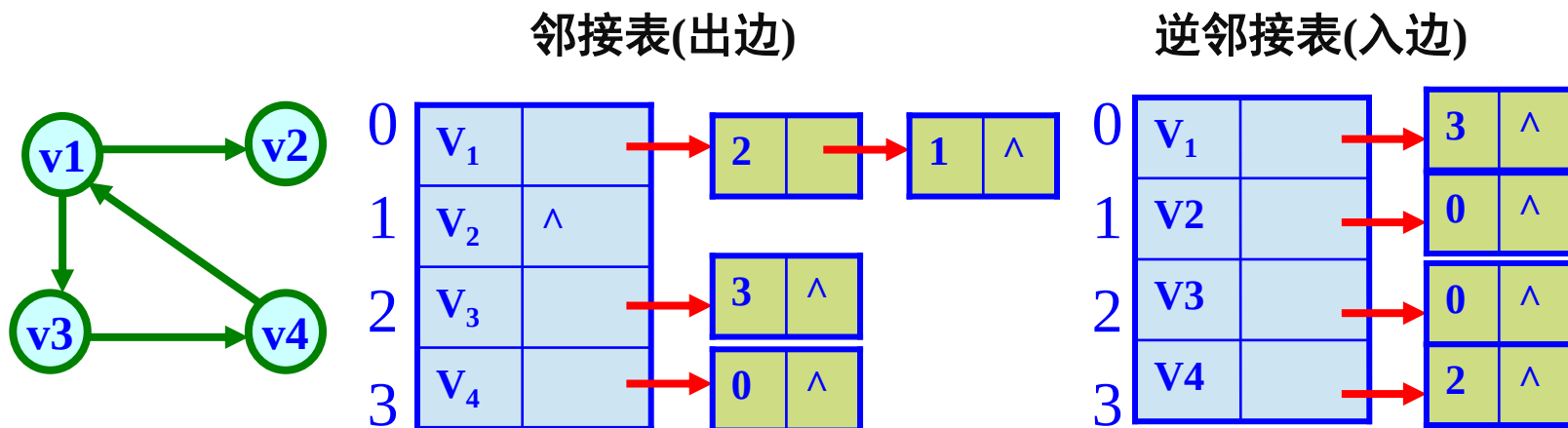
邻接表



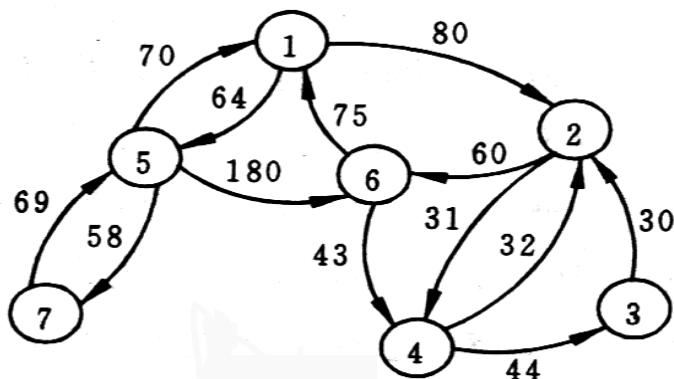
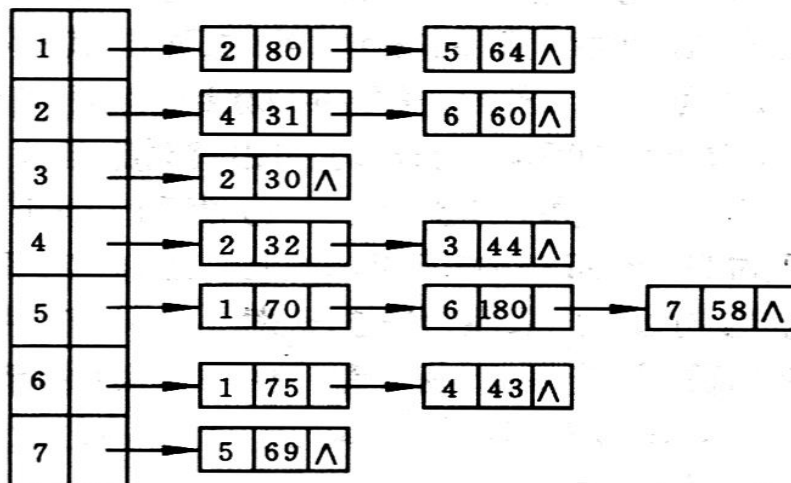
此无权图未开第3分量

请注意：邻接表不惟一！因各个边结点的链入顺序是任意的。

例2：有向图的邻接表如何表示？



例3：已知某网的邻接（出边）表，请画出该网络。



当邻接表的存储结构形成后，图便唯一确定！

邻接表存储法的特点：

—它其实是对邻接矩阵法的一种改进

分析1：对于 n 个顶点 e 条边的**无向图**，邻接表中除了 n 个头结点外，只有 $2e$ 个表结点，空间效率为 $O(n+2e)$ 。

若是稀疏图($e \ll n^2$)，则比邻接矩阵表示法 $O(n^2)$ 省空间。

怎样计算无向图顶点的度？ $TD(V_i)$ =单链表中链接的结点个数

分析2: 在**有向图**中, 邻接表中除了 n 个头结点外, 只有 e 个表结点, 空间效率为 $O(n + e)$ 。若是稀疏图, 则比邻接矩阵表示法合适。

怎样计算有向图顶点的出度? **$OD(V_i) = \text{单链出边表中链接的结点数}$**

怎样计算有向图顶点的入度? **$ID(V_i) = \text{邻接点为 } V_i \text{ 的弧个数}$**

怎样计算有向图顶点 V_i 的度: **$TD(V_i) = OD(V_i) + ID(V_i)$**

邻接表的优点： 空间效率高； 容易寻找顶点的邻接点；

邻接表的缺点： **判断两顶点间是否有边或弧**，需搜索两结点对应的单链表，没有邻接矩阵方便。

讨论： 邻接表与邻接矩阵有什么异同之处？

1. **联系：** 邻接表中每个链表对应于邻接矩阵中的一行，链表中结点个数等于一行中非零元素的个数。

2. 区别:

- ① 对于任一确定的**无向图**，邻接矩阵是唯一的（行列号与顶点编号一致），但**邻接表不唯一**（链接次序与顶点编号无关）。
- ② 邻接矩阵的空间复杂度为 $O(n^2)$ ，而邻接表的空间复杂度为 $O(n+e)$ 。

3. 用途:

邻接矩阵多用于稠密图的存储（ e 接近 $n(n-1)/2$ ）；
而邻接表多用于稀疏图的存储（ $e \ll n^2$ ）

图的邻接表在机内如何表示？

```
#define MAX_VERTEX_NUM 20 //假设的最大顶点数
```

```
Typedef struct ArcNode {    // 弧结构
    int adjvex;              // 该弧所指向的顶点位置
    struct ArcNode *nextarcs; // 指向下一条弧的指针
    InfoArc *info;           // 该弧相关信息的指针
} ArcNode;
```

```
Typedef struct VNode{    // 顶点结构
    VertexType data;      // 顶点信息
    ArcNode *firstarc;    // 指向依附该顶点的第一条弧的指针
}VNode, AdjList[ MAX_VERTEX_NUM ];
```

```
Typedef struct {    //图结构
    AdjList vertices; // 邻接表
    int vexnum, arcnum; // 顶点总数和弧总数
    int kind;         // 图的种类（用标志）
}ALGraph;
```

空间效率为 $O(n+2e)$ $O(n+e)$ ，时间效率为 $O(n+e*n)$

3. 十字链表表示法

它是有向图的另一种链式结构, 是邻接表、逆邻接表的结合。

思路: 将邻接矩阵用链表存储

- 1、弧结点, 设5个域 (每段弧是一个数据元素)
- 2、顶点结点, 设3个域 (每个顶点也是一个数据元素)

特点: 容易求得入度和出度

3. 十字链表表示法

弧结点

tailvex	headvex	hlink	tlink	info
---------	---------	-------	-------	------



tailvex: 弧尾顶点位置
headvex: 弧头顶点位置
hlink: 弧头相同的下一弧位置
tlink: 弧尾相同的下一弧位置
info: 弧信息

顶点结点

data	Firstin	Firstout
------	---------	----------



data : 顶点信息
Firstin : 以顶点为弧头的第一条弧结点
Firstout: 以顶点为弧尾的第一条弧结点

n 个顶点的集合怎样储存?

仍用顺序存储结构

4. 邻接多重表表示法

这是**无向图**的另一种存储结构，当**对边操作时**建议采用此种结构存储。

- 1、设立边结点， 6个域（每条边是一个数据元素）
- 2、设立顶点结点， 2个域（每个顶点也是一个数据元素）

特点：**同一条边，只用一个结点表达**

4. 邻接多重表表示法

边结点

mark	ivex	ilink	jvex	jlink	info
------	------	-------	------	-------	------



mark: 标志域
ivex, jvex: 边依附的两个顶点位置
ilink: 指向下一条依附顶点*i*的边位置
jlink: 指向下一条依附顶点*j*的边位置
info: 边信息

顶点结点

data	Firstedge
------	-----------



data: 存储顶点信息
Firstedge: 依附顶点的第一条边结点

n 个顶点的集合怎样储存?

仍用顺序存储结构

- <https://www.cs.usfca.edu/~galles/visualization/Algorithms.html>

7.1 图的基本概念

7.2 图的存储结构

7.3 图的遍历

7.4 图的连通性

7.5 图的应用

7.6 最短路径

遍历定义：从已给的连通图中某一顶点出发，沿着一些边，**访遍图中所有的顶点**，且使**每个顶点仅被访问一次**，就叫做**图的遍历**，它是图的基本运算。

图的特点：图中可能存在回路，且图的任一顶点都可能与其它顶点相通，在访问完某个顶点之后可能会沿着某些边又回到了曾经访问过的顶点。

如何避免重复访问？

设置一个辅助数组 $visited[n]$ ，用来标记每个被访问过的顶点。它的初始状态为0，在图的遍历过程中，一旦某一个顶点 i 被访问，就立即改 $visited[i]$ 为1，防止它被多次访问。

图常用的遍历：

- 一、深度优先搜索
- 二、广度优先搜索

深度优先搜索（遍历）步骤：

简单归纳：

- 访问起始点 v ；
- 若 v 的第1个邻接点没访问过，深度遍历此邻接点；
- 若当前邻接点已访问过，再找 v 的第2个邻接点重新遍历

深度优先搜索（遍历）步骤：

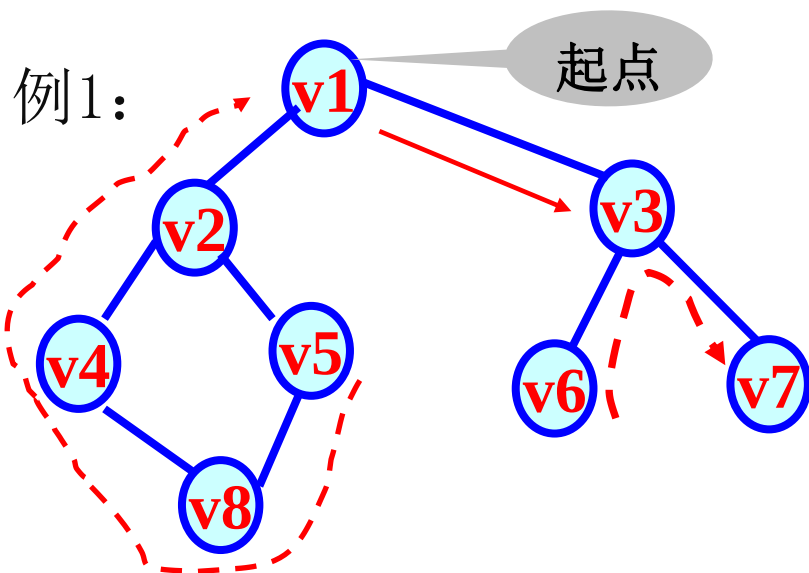
详细归纳：

1. 在访问图中某一起始顶点 v 后，由 v 出发，访问它的任一邻接顶点 w_1 ；再从 w_1 出发，访问与 w_1 邻接但还未被访问过的顶点 w_2 ；然后再从 w_2 出发，进行类似的访问，直至到达所有的邻接顶点都被访问过的顶点 u 为止。
2. 接着，退回一步，退到前一次刚访问过的顶点，看是否还有其它未被访问的邻接顶点。如果有则访问此顶点，之后从此顶点出发，进行与前述类似访问；如果没有，就再退回一步进行搜索。重复上述过程，直到连通图中所有顶点都被访问过为止。

一、深度优先搜索(DFS)

Depth_First Search

基本思想：一仿树的先序遍历过程。



DFS结果

$v1 \rightarrow v2 \rightarrow v4 \rightarrow v8 \rightarrow$
 $v5 \rightarrow v3 \rightarrow v6 \rightarrow v7$

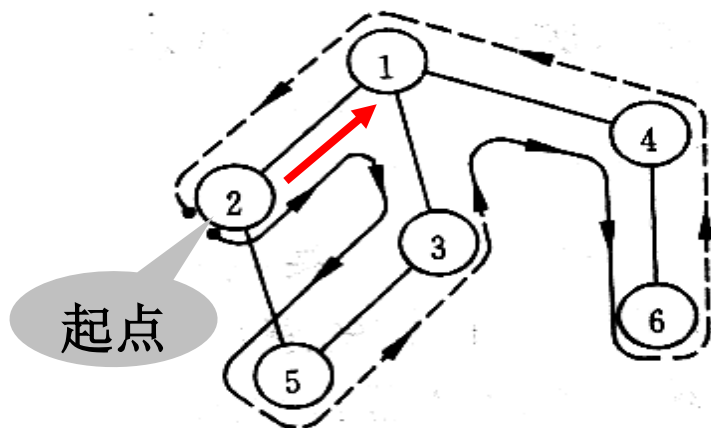
应退回到v1, 因为v2已有标记

一、深度优先搜索(DFS)

Depth_First Search

基本思想：一仿树的先序遍历过程。

例2:



DFS结果

$v_2 \rightarrow v_1 \rightarrow v_3 \rightarrow v_5 \rightarrow$

$v_4 \rightarrow v_6$

应退回到V1, 因为V3已有标记

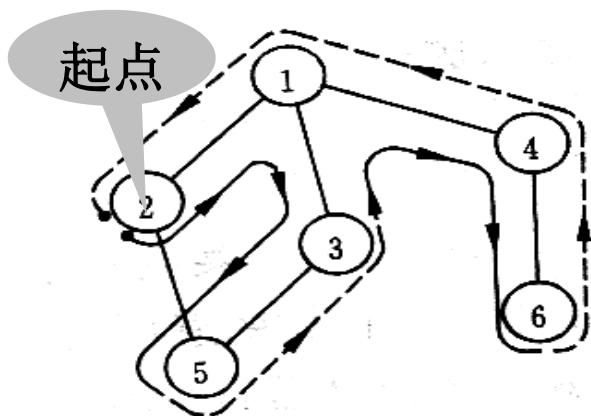
7.3

图的遍历

例：

邻接矩阵 A

	1	2	3	4	5	6
1	0	1	1	1	0	0
2	1	0	0	0	1	0
3	1	0	0	0	1	0
4	1	0	0	0	0	1
5	0	1	1	0	0	0
6	0	0	0	1	0	0



辅助数组 $visited[n]$

	1	2	3	4	5	6
1	0	0	1	1	1	1
2	0	1	1	1	1	1
3	0	0	0	1	1	1
4	0	0	0	0	0	1
5	0	0	0	0	1	1
6	0	0	0	0	0	1

DFS 结果

$v2 \rightarrow v1 \rightarrow v3 \rightarrow v5 \rightarrow v4 \rightarrow v6$

请注意逐级回退是递归概念

讨论2：DFS算法如何编程？——可以用递归算法！

```
Boolean visited[MAX];
Status (*VisitFunc)(int v); // 全局函数变量
void DFSTraverse(Graph G, Status (*Visit)(int v)) {
    /* 对图G作深度优先遍历 */
    VisitFunc = Visit;
    for (v=0; v<G.vexnum; ++v)
        visited[v] = FALSE; // 访问标志数组初始化
    for (v=0; v<G.vexnum; ++v)
        if (!visited[v]) // 对尚未访问的顶点调用DFS
            DFS(G, v);
}
```

算法7.4

讨论2: DFS算法如何编程? ——可以用递归算法!

```
void DFS(Graph G, int v) {  
    /* 从顶点v出发, 深度优先搜索遍历连通图G */  
    visited[v] = TRUE;  
    VisitFunc(v); //访问第v个顶点  
    for( w=FirstAdjVex(G,v); w>=0 ; w=NextAdjVex(G,v,w) )  
        if (!visited[w])  
            DFS(G, w); // 对v的尚未访问的邻接顶点w 递归调用DFS  
} // DFS
```

算法 7.5

DFS 算法效率分析:

(设图中有 n 个顶点, e 条边)

- 如果用邻接矩阵来表示图, 遍历图中每一个顶点都要从头扫描该顶点所在行, 因此遍历全部顶点所需的时间为 $O(n^2)$ 。
- 如果用邻接表来表示图, 遍历图的时间复杂度为 $O(n+e)$ 。

结论:

稠密图适于在邻接矩阵上进行深度遍历;

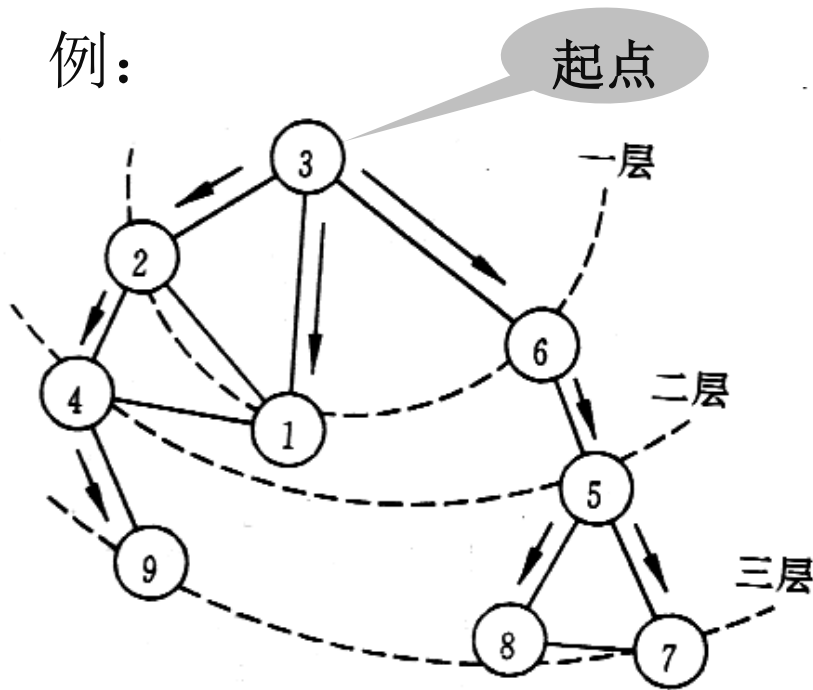
稀疏图适于在邻接表上进行深度遍历。

二、广度优先搜索(BFS)

Breadth_First Search

基本思想：仿树的**层次遍历**过程。

例：

**BFS 结果**

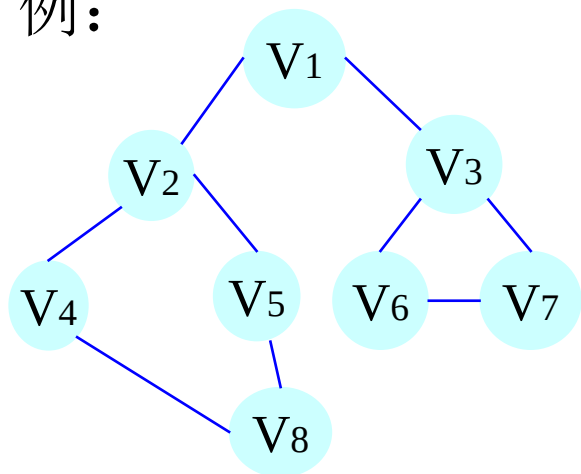
$v_3 \rightarrow v_2 \rightarrow v_1 \rightarrow v_6 \rightarrow$
 $v_4 \rightarrow v_5 \rightarrow v_9 \rightarrow v_8 \rightarrow v_7$

广度优先搜索（遍历）步骤：

简单归纳：

- 在访问了起始点 v 之后，依次访问 v 的邻接点；
- 然后再依次（顺序）访问这些点（下一层）中未被访问过的邻接点；
- 直到所有顶点都被访问过为止。

例：



广度优先遍历（结果不唯一）：

$$V_1 \Rightarrow V_2 \Rightarrow V_3 \Rightarrow V_4 \Rightarrow V_5 \Rightarrow V_6 \Rightarrow V_7 \Rightarrow V_8$$
$$V_1 \Rightarrow V_3 \Rightarrow V_2 \Rightarrow V_7 \Rightarrow V_6 \Rightarrow V_5 \Rightarrow V_4 \Rightarrow V_8$$
$$V_1 \Rightarrow V_2 \Rightarrow V_3 \Rightarrow V_5 \Rightarrow V_4 \Rightarrow V_7 \Rightarrow V_6 \Rightarrow V_8$$

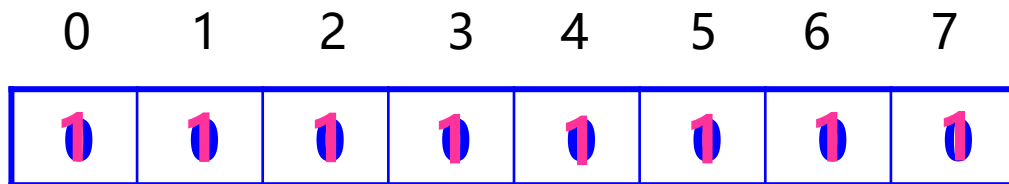
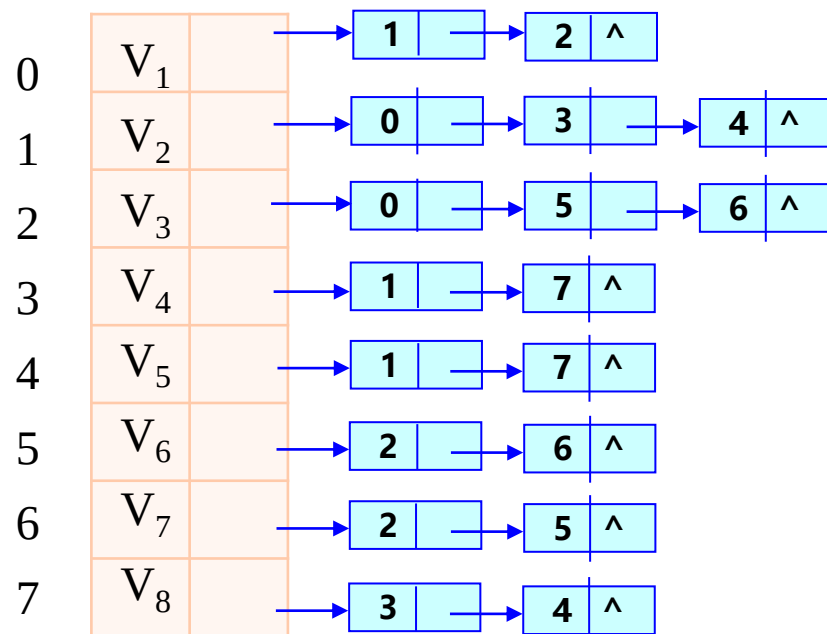
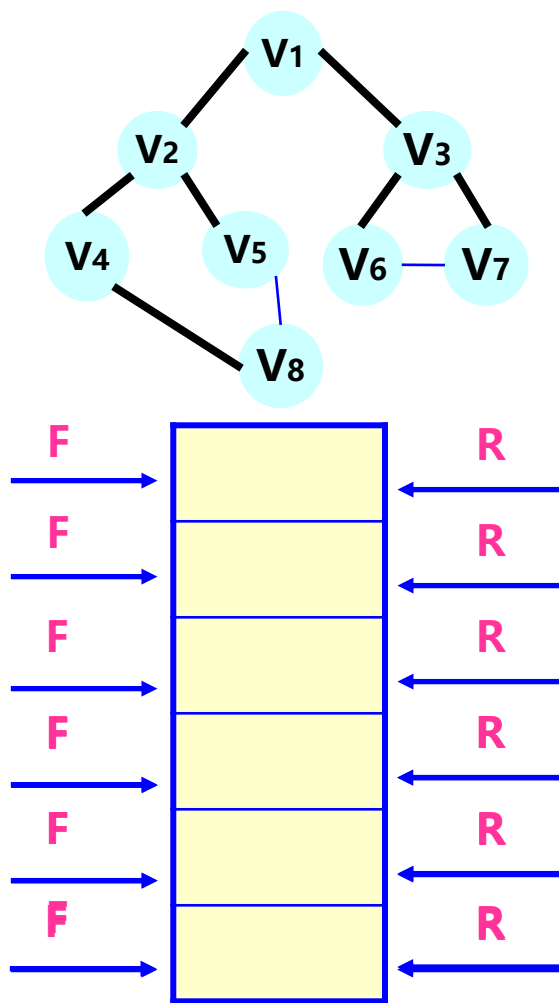
广度优先搜索是一种分层的搜索过程，每向前走一步可能访问一批顶点，不像深度优先搜索那样有回退的情况。

因此，广度优先搜索不是一个递归的过程，其算法也不是递归的。

7.3

图的遍历

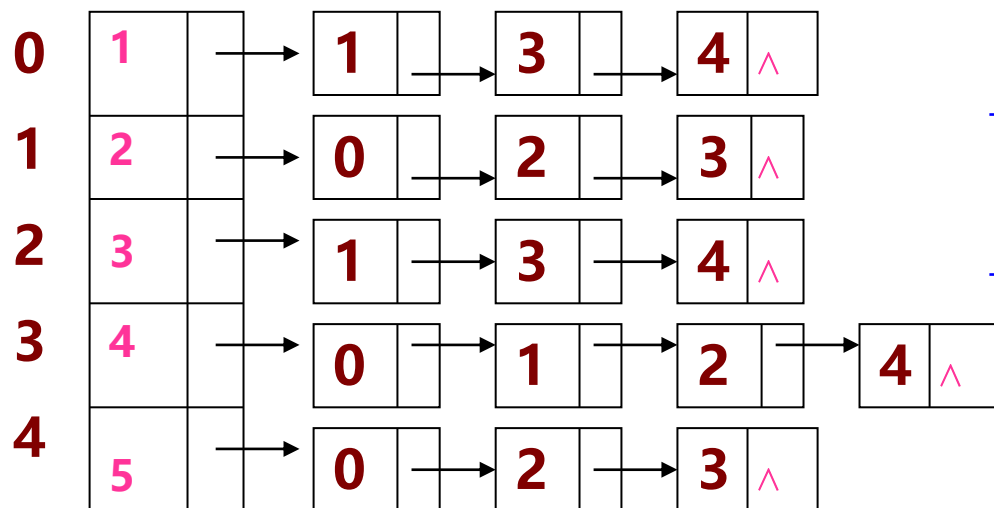
利用队列实现广度优先遍历算法 —— 使用队列进行层次遍历！



$V_1 \Rightarrow V_2 \Rightarrow V_3 \Rightarrow V_4 \Rightarrow V_5 \Rightarrow V_6 \Rightarrow V_7 \Rightarrow V_8$

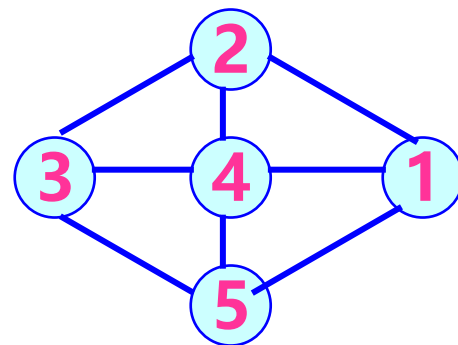
BFS算法（算法7.6）

```
void BFSTraverse(Graph G, Status (* Visit) {  
    for (v=0; v<G.vexnum; ++v) visited[v] = FALSE; //辅助数组  
    InitQueue(Q); //置空的辅助队列Q  
    for (v=0; v<G.vexnum; ++v) //最外层大循环，每个顶点都要搜索  
        if (!visited[v]) { //v尚未访问  
            visited[v]=TRUE; visit(v);  
            EnQueue(Q, v); //v入队列  
            while (!QueueEmpty(Q)) {  
                DeQueue(Q, u); //队头元素出队，记为u  
                for (w=FirstAdjVex(G,u); w; w=NextAdjVex(G,u,w))  
                    if (!visited[w]) { //w为u的尚未访问的邻接顶点  
                        visited[w] = TRUE; Visit(w);  
                        EnQueue(Q,w);  
                    }  
            }  
        }  
    }  
}
```



队头

队尾



调用bfs()的执行过程如下(设初始结点v=3)

访问结点3，入队。

第一次循环：结点3出队，找其第一个邻接点2，2未访问过，访问之并入队；找下一个邻接点4，访问之并入队；找下一个邻接点5，访问之并入队

第二次循环：结点2出队，找其第一个邻接点1，访问、入队；下一个邻接点3，访问过；下一个邻接点4，访问过。

第三次循环：结点4出队，依次找其邻接点1、2、3、5均访问过

第四次循环：结点5出队，依次找其邻接点1、3、4均访问过

第五次循环：结点1出队，依次找其邻接点2、4、5均访问过

此时队列为空，遍历结束，遍历序列为：3、2、4、5、1

遍历算法总结

1. 遍历图的过程实质上是**通过边或弧找邻接点**的过程，因此DFS和BFS的时间复杂度是相同的。
2. 对于一个图，它的DFS和BFS的搜索顺序不唯一，**是由该图的存储结构决定的**。
3. 如果从无向图的任一顶点出发进行一次深度优先搜索即可访问所有顶点，则该图一定是连通图。
4. 对于非连通图的遍历过程中每调用一次DFS算法都得到该图的一个连通分量。

7.1 图的基本概念

7.2 图的存储结构

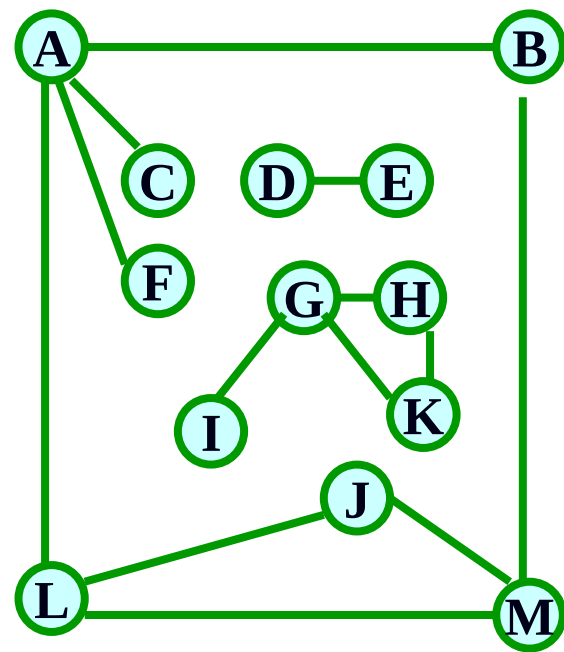
7.3 图的遍历

7.4 图的连通性

7.5 图的应用

7.6 最短路径

连通图：在无向图中，若从顶点 v_1 到顶点 v_2 有路径，则称顶点 v_1 与 v_2 是**连通的**。如果图中任意一对顶点都是连通的，则称此图是**连通图**。非连通图的极大连通子图叫做**连通分量**。



强连通图：在有向图中，若对于每一对顶点 v_i 和 v_j ，都存在一条从 v_i 到 v_j 和从 v_j 到 v_i 的路径，则称此图是**强连通图**。

连通分量 (Connected component)

- 当**无向图为非连通图时**，从图中某一顶点出发，利用深度优先搜索算法或广度优先搜索算法**不可能遍历到图中的所有顶点**，只能访问到该顶点所在的最大连通子图(连通分量)的所有顶点。
- 若从无向图的每一个连通分量中的一个顶点出发进行遍历，可求得无向图的所有连通分量。
- 在算法中，需要对图的每一个顶点进行检测：若已被访问过，则该顶点一定是落在图中已求得的连通分量上；若还未被访问，则从该顶点出发遍历图，可求得图的另一个连通分量。

生成树：是一个极小连通子图，它含有图中全部 n 个顶点，但只有 $n-1$ 条边。

如果在生成树上添加1条边，必定构成一个环。

若图中有 n 个顶点，却少于 $n-1$ 条边，必为非连通图。

- 使用不同的遍历图的方法，可以得到不同的生成树；从不同的顶点出发，也可能得到不同的生成树。
- 按照生成树的定义， n 个顶点的连通网络的生成树有 n 个顶点、 $n-1$ 条边。
- 构造生成树的准则
 - 必须只使用该网络中的边来构造生成树；
 - 必须使用且仅使用 $n-1$ 条边来联结网络中的 n 个顶点；
 - 不能使用产生回路的边。

问题：

假设要在 n 个城市之间建立通讯联络网，则连通 n 个城市只需要修建 $n-1$ 条线路，如何在最节省经费的前提下建立这个通讯网？

先建立数学模型：

顶点：表示城市，有 n 个；

边：表示线路，有 $n-1$ 条；

边的权值：表示线路的经济代价；

连通网：表示 n 个城市间的通信网。

显然此连通网
是一棵**生成树**！

问题抽象：

n 个顶点的生成树很多，需要从中选一棵**代价最小的生成树**，即该树各边的代价之和最小。此树便称为最小生成树(Minimum cost Spanning Tree, MST)。

求MST有多种算法，但最常用的是以下两种：

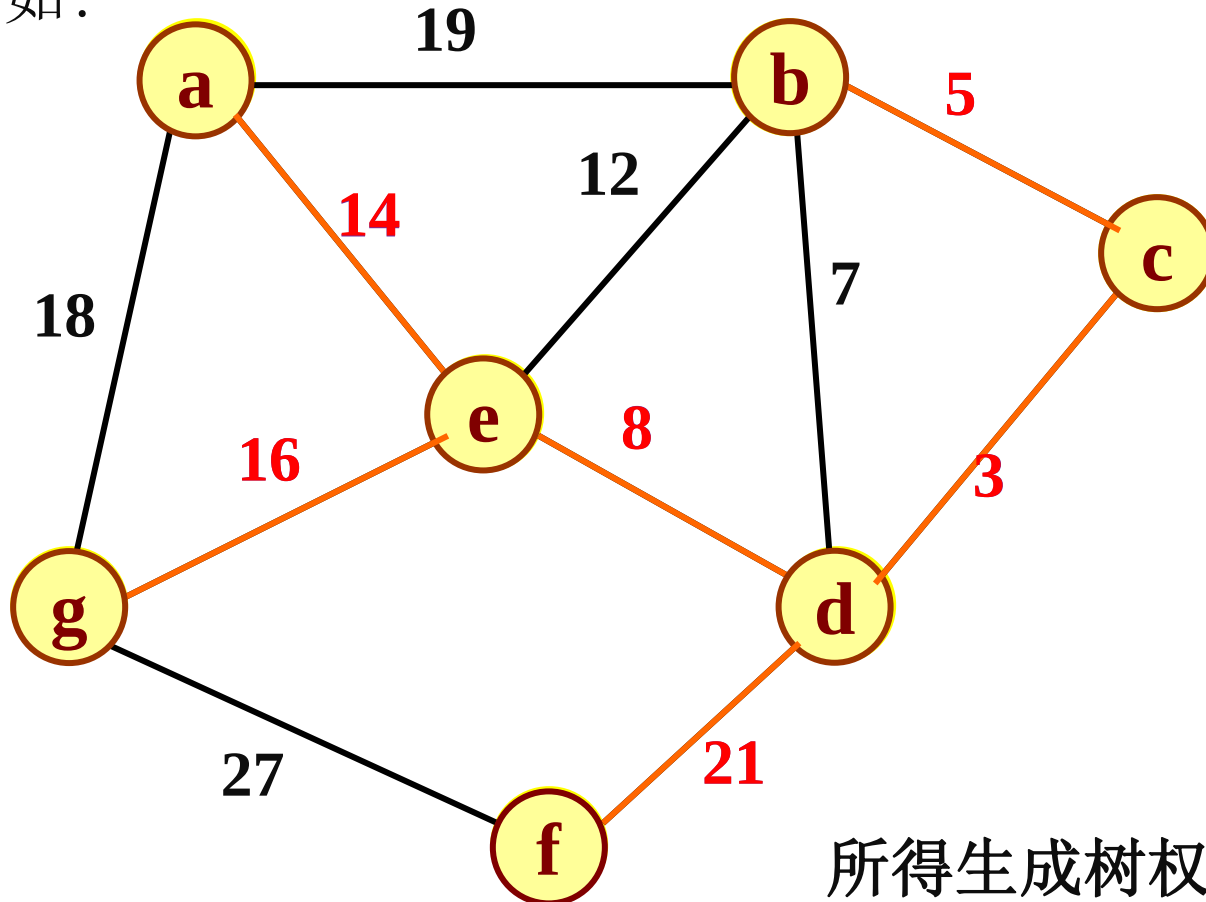
Prim（普里姆）算法：将顶点归并，与边数无关，适于稠密网。

Kruskal（克鲁斯卡尔）算法：将边归并，适于求稀疏网的最小生成树。

普里姆(Prim)算法:

取图中任意一个顶点 v 作为生成树的根，之后往生成树上添加新的顶点 w 。在添加的顶点 w 和已经在生成树上的顶点 v 之间必定存在一条边，并且该边的权值在所有连通顶点 v 和 w 之间的边中取值最小。之后继续往生成树上添加顶点，直至生成树上含有 n 个顶点为止。

例如：



所得生成树权值和
 $= 14 + 8 + 3 + 5 + 16 + 21 = 67$

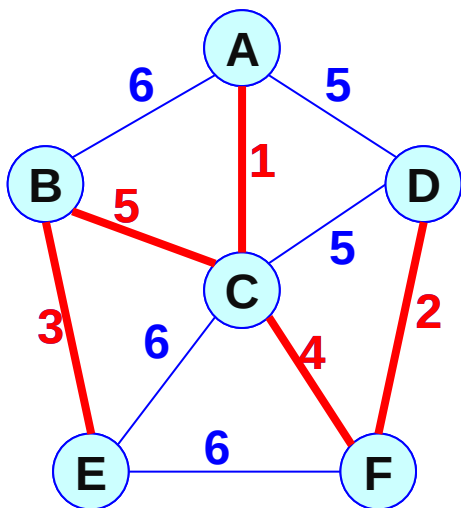
一般情况下所添加的顶点应满足下列条件:

在生成树的构造过程中, 图中 n 个顶点分属两个集合:

已落在生成树上的顶点集 U 和尚未落在生成树上的顶点集 $V-U$,

则应在所有连通 U 中顶点和 $V-U$ 中顶点的边中选取权值最小的边。

普里姆(Prim)算法:



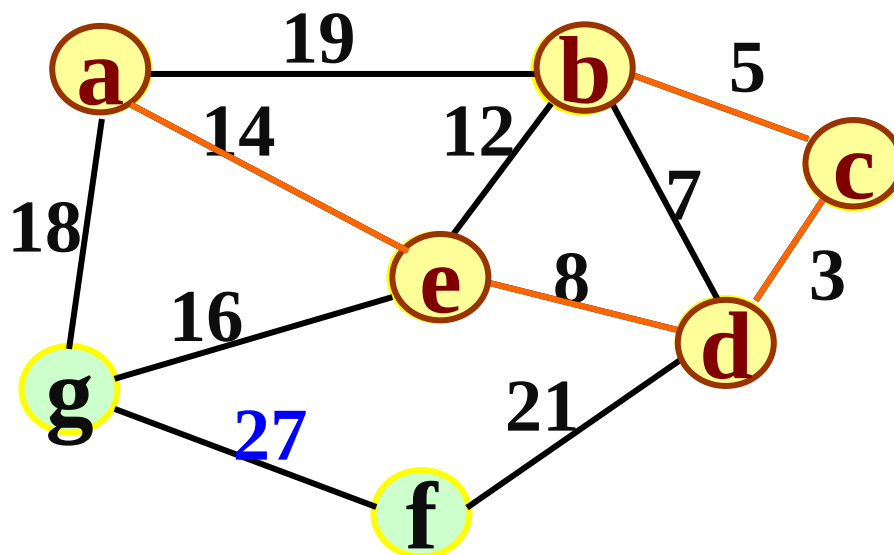
U	V-U	B	C	D	E	F	TE
{A}	{B,C,D,E,F}	AB 6	AC 1	AD 5	~ ∞	~ ∞	AC 1
{A,C}	{B,D,E,F}	CB 5		AD 5	CE 6	CF 4	CF 4
{A,C,F}	{B,D,E}	CB 5		FD 2	CE 6		FD 2
{A,C,F,D}	{B,E}	CB 5			CE 6		CB 5
{A,C,F,D,B}	{E}				BE 3		BE 3
{A,C,F,D,B,E}	{}						

普里姆(Prim)算法的实现:

设置一个辅助数组, 对当前V-U集中的每个顶点,
记录和顶点集U中顶点相连接的代价最小的边:

```
struct {  
    VertexType adjvex; // U集中的顶点序号  
    VRType lowcost; // 边的权值  
} closedge[MAX_VERTEX_NUM];
```

例如:



closededge \	0 (a)	1 (b)	2 (c)	3 (d)	4 (e)	5 (f)	6 (g)
Adjvex		c	d	e	a	d	e
Lowcost		5	3	8	14	21	16

普里姆(Prim)算法的实现（教材算法7.9）：

```
void MiniSpanTree_PRIM(MGraph G, VertexType u) {  
    //用Prim算法从第u个顶点出发构造网G的最小生成树T。  
    struct {  
        VertexType adjvex;  
        VRType lowcost;  
    } closedge[MAX_VERTEX_NUM];  
  
    k=LocateVex(G,u);  
  
    for (j=0; j<G.vexnum; ++j) //辅助数组初始化  
        if ( j!=k) // adjvex, lowcost  
            closedge[j]={u, G.arcs[k][j].adj;}  
    closedge[k].lowcost = 0;
```

普里姆(Prim)算法的实现（教材算法7.9）：

```
for (i=1; i<G.vexnum; ++i) //选择其余G.vexnum-1个顶点;
    k=minimum(closedge); //求出T的下一个节点
    // 此时closedge[k].lowcost =
    // MIN{closedge[vi].lowcost | closedge[vi].lowcost>0, vi ∈ V-U}.
    printf(closedge[k].adjvex, G.vexs[k]); //输出生成树的边
    closedge[k].lowcost = 0; // 第k顶点并入U集
    for (j=0; j<G.vexnum; ++j)
        // 新顶点并入U后重新选择最小边
        if (G.arc[k][j].adj<closedge[j].lowcost)
            closedge[j]={ G.vexs[k], G.arcs[k][j].adj};
    }//for i
} //MiniSpanTree
```

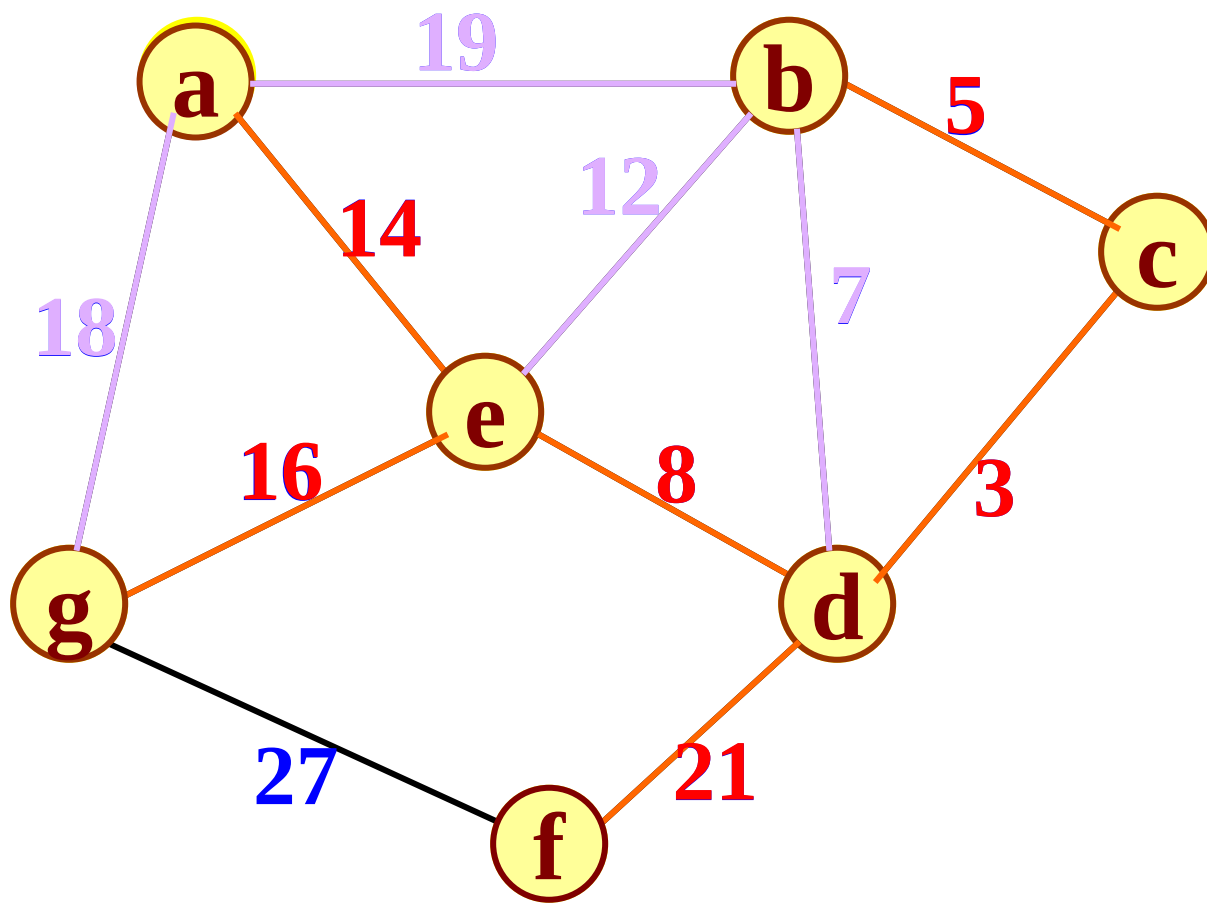

克鲁斯卡尔（Kruskal）算法：

为使生成树上边的权值之和达到最小，则应使生成树中每一条边的权值尽可能地小。

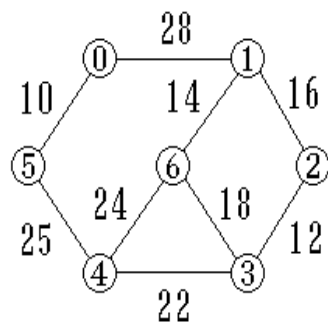
具体做法：

先构造一个只含 n 个顶点的子图SG，然后从权值最小的边开始，若它的添加不使SG中产生回路，则在SG上加上这条边，如此重复，直至加上 $n-1$ 条边为止。

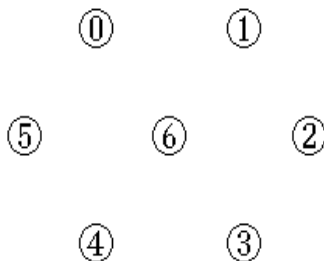
例如:



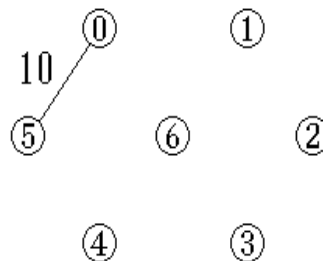
应用克鲁斯卡尔算法构造最小生成树的过程



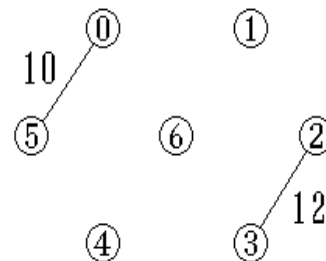
(a)



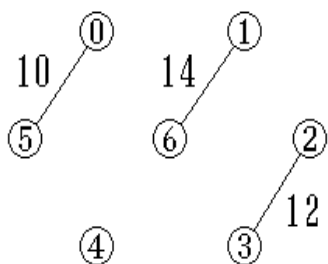
(b)



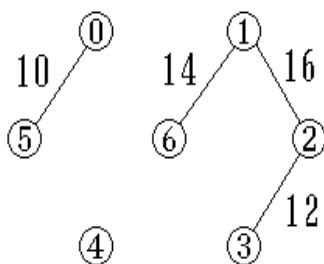
(c)



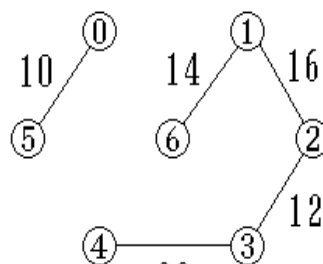
(d)



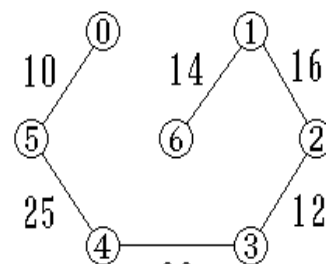
(e)



(f)



(g)



(h)

比较两种算法

	Prim算法	Kruskal算法
时间复杂度	$O(n^2)$	$O(e \log e)$
适应范围	稠密图	稀疏图

n : 顶点数, e : 边数

7.1 图的基本概念

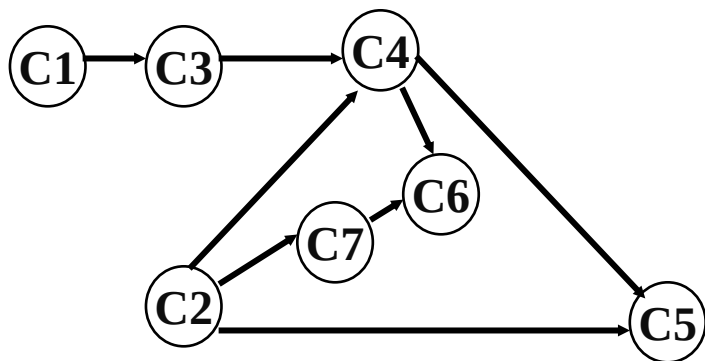
7.2 图的存储结构

7.3 图的遍历

7.4 图的连通性

7.5 图的应用

7.6 最短路径



用顶点表示活动，用弧表示活动间的优先关系的有向图称为**Activity On Vertex Network (AOV-网)**

举例：一个计算机技术应用专业的学生必须学习的基础课程有：

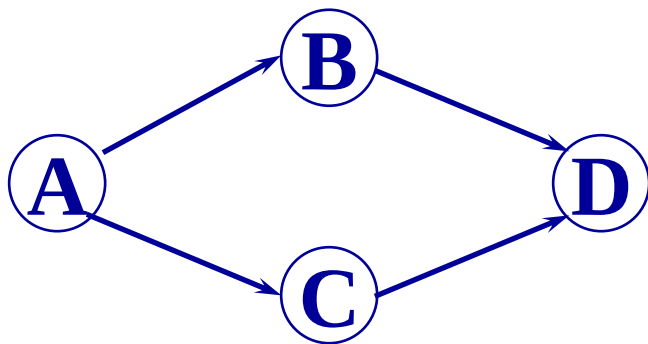
课程代号	课程名称	先修课程
C1	高等数学	无
C2	程序设计	无
C3	离散数学	C1
C4	数据结构	C2 C3
C5	编译原理	C2 C4
C6	操作系统	C4 C7
C7	计算机组成原理	C2

学习顺序：
 c1 c3 c2 c4 c7 c6 c5
 c2 c7 c1 c3 c4 c5 c6

拓扑排序

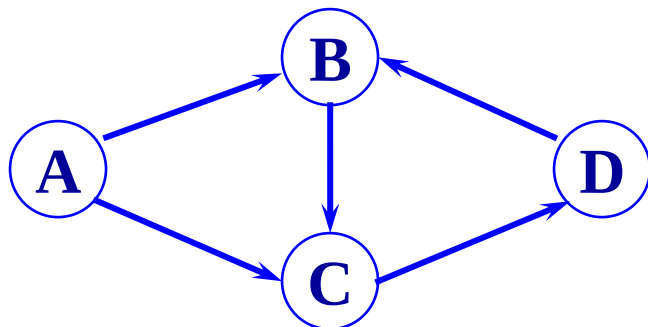
按照有向图给出的次序关系，将图中顶点排成一个线性序列，对于有向图中没有限定次序关系的顶点，则可以人为加上任意的次序关系。由此所得顶点的线性序列称之为拓扑有序序列

例如：对于下列有向图



可求得拓扑有序序列：

A B C D 或 A C B D

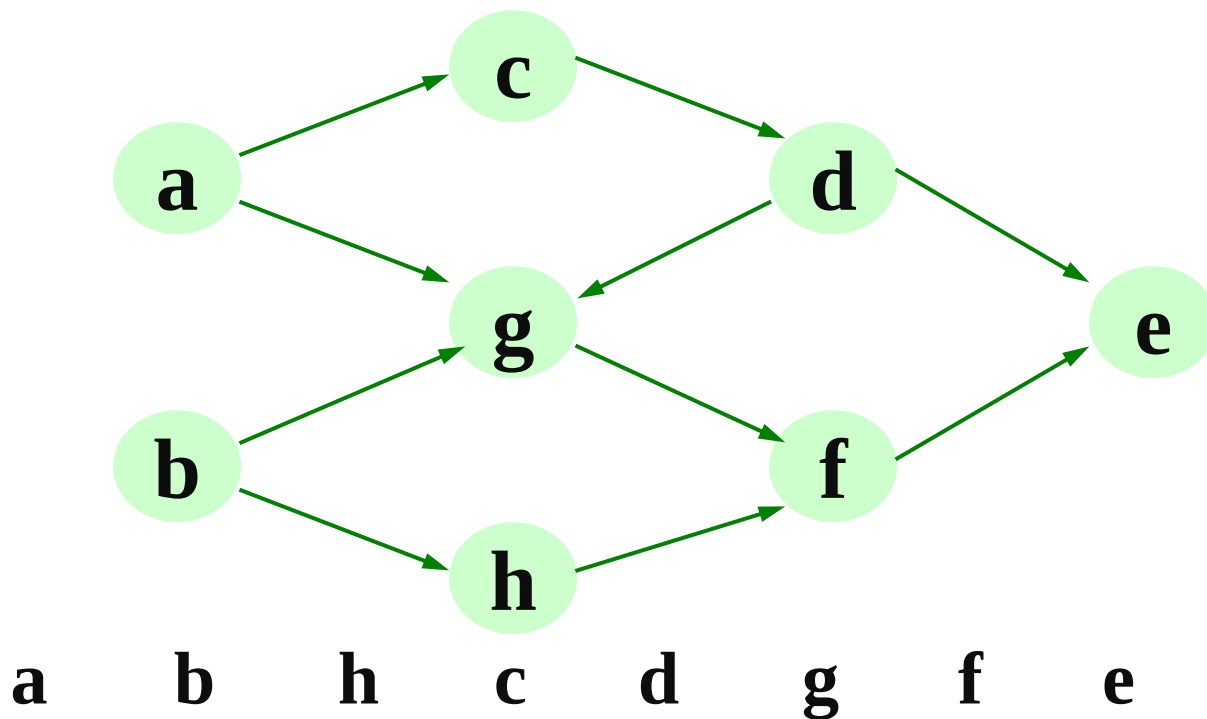


不能求得它的拓扑有序序列。

因为图中存在一个回路{B, C, D}

如何进行拓扑排序？

- 一、从有向图中选取一个没有前驱的顶点，并输出之；
 - 二、从有向图中删去此顶点以及所有以它为尾的弧；
- 重复上述两步，直至图空，或者图不空但找不到无前驱的顶点为止。



没有前驱的顶点 = 入度为零的顶点

删除顶点及以它为尾的弧 = 弧头顶点的入度减1

采用邻接表作有向图的存储结构，且在头结点中增加一个存放顶点入度的数组(indegree)。为避免每次都要搜索入度为零的顶点，在算法中设置一个“栈”，以保存“入度为零”的顶点。

```
Status TopologicalSort(ALGraph G) {  
    // 有向图G采用邻接表存储结构。  
    // 若G无回路，则输出G的顶点的一个拓扑序列并返回OK，  
    // 否则ERROR。  
    FindInDegree(G, indegree); //对各顶点求入度  
    InitStack(S); //建零入度顶点栈S  
    for ( i=0; i<G.vexnum; ++i)  
        if (!indegree[i]) Push(S, i); //入度为零的顶点入栈
```

```
count=0;           //对输出顶点计数
while (!EmptyStack(S)) {
    //输出i号顶点并计数。
    Pop(S, i); ++count; printf(i,G.vertices[i].data);
    for (p= G.vertices[i]. Firstarc; p; p=p->Nextarc){
        k=p->adjvex; // 对i号顶点的每个邻接点的入度减1
        if (!(--indegree[k])) Push(S, k); //若入度为零，则入栈
    }//for
} //while
if (count<G.vexnum) return ERROR ; //该图中有回路
else return OK;
} //TopologicalSort
```

算法 7.12

算法复杂度分析：

对有 n 个顶点和 e 条弧的有向图，建立求各顶点的入度的时间复杂度为 $O(e)$ ；建零入度顶点栈的时间复杂度为 $O(n)$ 。在拓扑排序中，若有向图无环，则每个顶点进一次栈，出一次栈，入度减1的操作在while语句中共执行 e 次。所以，总的时间复杂度为 $O(n+e)$ 。

7.1 图的基本概念

7.2 图的存储结构

7.3 图的遍历

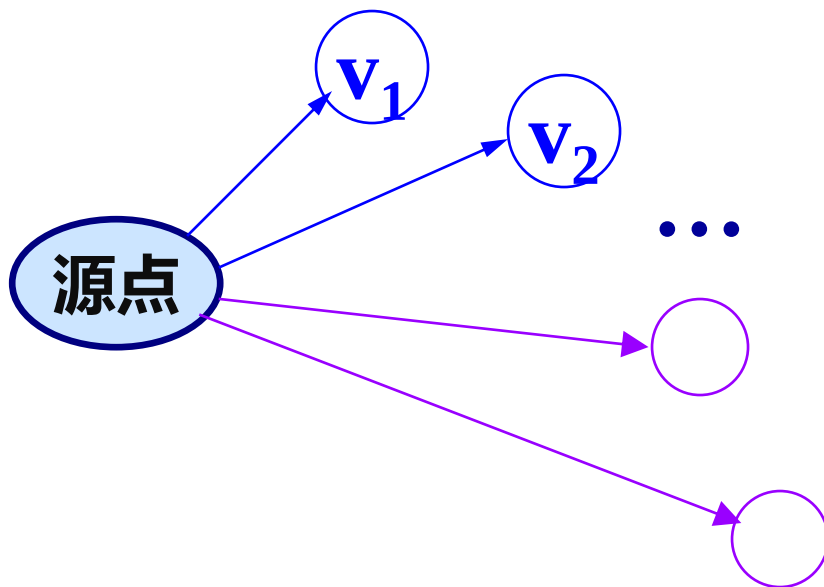
7.4 图的连通性

7.5 图的应用

7.6 最短路径

求从源点到其余各点的最短路径:

依最短路径的长度递增的次序求得各条路径。假设为 $\langle v_0, v_1 \rangle$, $\langle v_0, v_1, v_2 \rangle, \dots$



其中，从源点到顶点 v_1 最短路径是所有最短路径中长度最短者。

路径长度最短的最短路径的特点：

在这条路径上，必定只含一条弧，并且这条弧的权值最小。

下一条路径长度次短的最短路径的特点：

它只可能有两种情况：或者是直接从源点到该点（只含一条弧）；或者是从源点经过顶点 v_1 ，再到达该顶点（由两条弧组成）。

再下一条路径长度次短的最短路径的特点:

它可能有三种情况：或者是直接从源点到该点(只含一条弧)；或者是从源点经过顶点 v_1 ，再到达该顶点(由两条弧组成)；或者是从源点经过顶点 v_2 ，再到达该顶点。

其余最短路径的特点:

它或者是**直接从源点到该点**(只含一条弧)；或者是**从源点经过已求得最短路径的顶点**，再到达该顶点。

求最短路径的迪杰斯特拉 (Dijkstra) 算法:

设置辅助数组 $Dist$,其中每个分量 $Dist[k]$ 表示当前所求得的从源点到其余各顶点 k 的最短路径。

一般情况下,

$Dist[k] = \langle \text{源点到顶点} k \text{的弧上的权值} \rangle$

或者 $= \langle \text{源点到其它顶点的路径长度} \rangle$

+ $\langle \text{其它顶点到顶点} k \text{的弧上的权值} \rangle$ 。

1) 在所有从源点出发的弧中选取一条权值最小的弧，即为第一条最短路径。

$$Dist[k] = \begin{cases} G.arcs[v0][k] & v0 \text{ 和 } k \text{ 之间存在弧} \\ INFINITY & v0 \text{ 和 } k \text{ 之间不存在弧} \end{cases}$$

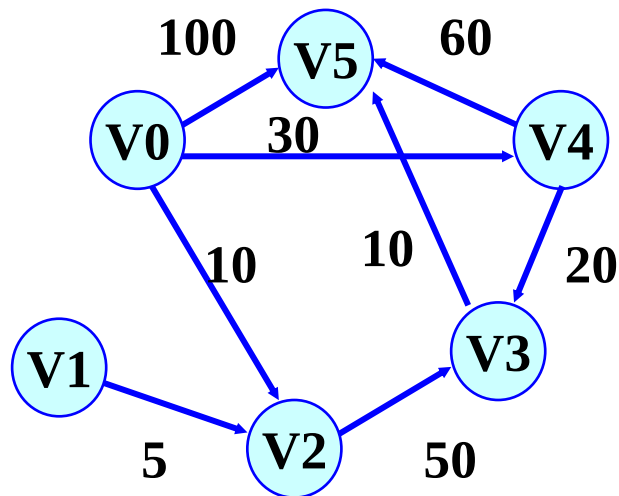
其中的最小值即为最短路径的长度

2) 修改其它各顶点的 $Dist[k]$ 值。

假设求得最短路径的顶点为 u ，若 $Dist[u] + G.arcs[u][k] < Dist[k]$ ，则将 $Dist[k]$ 改为 $Dist[u] + G.arcs[u][k]$ 。

7.6

最短路径问题



	0	1	2	3	4	5
0	∞	∞	10	∞	30	100
1	∞	∞	5	∞	∞	∞
2	∞	∞	∞	50	∞	∞
3	∞	∞	∞	∞	∞	10
4	∞	∞	∞	20	∞	60
5	∞	∞	∞	∞	∞	∞

终点	求解过程				
	1	2	3	4	5
V1	∞	∞	∞	∞	∞
V2	10 (V0,V2)				
V3	∞	60 (V0,V2, V3)	50 (V0,V4, V3)		
V4	30 (V0,V4)	30 (V0,V4)			
V5	100 (V0,V5)	100 (V0,V5)	90 (V0,V4, V5)	60 (V0,V4, V3,V5)	
Vj	V2	V4	V3		
S	V0,V2	V0,V2, V4	V0,V2, V4, V3	V0,V2, V4, V3, V5	

Dijkstra算法可描述如下：

- (1) 初始化： $S \leftarrow \{ v_0 \}$;
 $Dist[j] \leftarrow Edge[0][j]$, $j = 1, 2, \dots, n-1$;
// n 为图中顶点个数
- (2) 求出最短路径的长度：
 $dist[k] \leftarrow \min\{ dist[i] \}, i \in V - S$;
 $S \leftarrow S \cup \{ k \}$;
- (3) 修改：
 $dist[i] \leftarrow \min\{ dist[i], dist[k] + Edge[k][i] \}$,
对于每一个 $i \in V - S$;
- (4) 判断：若 $S = V$ ，则算法结束，否则转(2)。

Dijkstra算法: 教材p189算法7.15

```
void Shortest_DIJ(MGraph G, int v0, PathMatrix &P, ShortPathTable &D)
{ // 用Dijkstra算法求有向图G的v0顶点到其余顶点v的最短路径
  // P[v]及其带权路径长度D[v]。若P[v][w]为TRUE, 则w是从v0
  // 到v当前求得最短路径上的顶点。final[v]为TRUE当且仅当
  //  $v \in S$ , 即求得从v0到v的最短路径
  for (v=0; v<G.vexnum; ++v){
    final[v] = FALSE; D[v] = G.arcs[v0][v];
    for (w=0; w<G.vexnum; ++w) P[v][w] = FALSE; // 设空路径
    if (D[v]<INFINITY) { P[v][v0] = TRUE; P[v][v] = TRUE; }
  } // for
  D[v0]=0; final[v0]=TRUE; // 初始化, v0顶点属于S集
} // Shortest_DIJ
```

Dijkstra算法: 教材p189算法7.15

//开始主循环, 每次求得v0到某个v顶点的最短路径, 并加v到S集

for (i=1; i<G.vexnum; ++i) { //其余G.vexnum-1个顶点

min = INFINITY; //当前所知离v0顶点的最近距离

for (w=0; w<G.vexnum; ++w)

if (!final[w]) // w顶点在V-S中

if (D[w]<min) { v=w; min = D[w] } //w顶点离v0顶点更近

final[v] = TURE; //离v0顶点最近的v加入S集

(2)

for (w=0; w<G.vexnum; ++w) //更新当前最短路径及距离

if (!final[w] && (min+G.arcs[v][w]<D[w])) {

//修改D[w]和P[w], $w \in V-S$

D[w] = min + G.arcs[v][w];

P[w] = P[v]; P[w][w] = TURE; // $P[w]=P[v]+[w]$

(3)

}//for

}//Shortest_DIJ

1. 掌握图的概念和术语；熟悉图的存储结构及其构造算法，了解实际问题的求解效率与采用何种存储结构有密切联系。
2. 熟练掌握图的两搜索路径的遍历：遍历的逻辑定义、深度优先搜索和广度优先搜索的算法。
3. 注意图的遍历算法与树的遍历算法之间的类似和差异。
4. 应用图的遍历算法求解各种简单路径问题（Prim ,Kruskal, 拓扑排序，Dijkstra）。
5. 理解教科书中讨论的各种图的算法。

- 7.1 (1) - (3)
- 7.3
- 7.11
- 7.22